Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский университет ИТМО»

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №2

Вариант – 772

Выполнил:

Козаченко Данил Александрович

Группа P3212

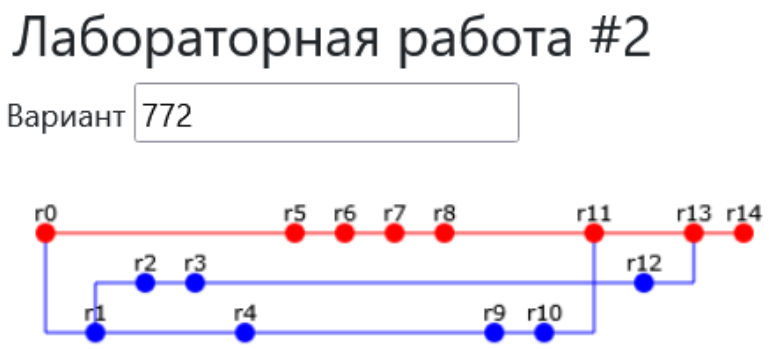
Проверил:

Райла Мартин

Санкт-Петербург 2026

Задание 3
Дополнительное задание 3
Список команд, использованных при создании и конфигурации репозиториев в домашнем каталоге пользователя 5
Номера ревизий и соответствующие им последовательности команд с комментариями 5
Выводы по работе 5
Приложение 1 5

Задание



Сконфигурировать в своём домашнем каталоге репозитории svn и git и загрузить в них начальную ревизию файлов с исходными кодами (в соответствии с выданным вариантом).

Воспроизвести последовательность команд для систем контроля версий svn и git, осуществляющих операции над исходным кодом, приведённые на блок-схеме.

При составлении последовательности команд необходимо учитывать следующие условия:

- Цвет элементов схемы указывает на пользователя, совершившего действие (красный - первый, синий - второй).
- Цифры над узлами - номер ревизии. Ревизии создаются последовательно.
- Необходимо разрешать конфликты между версиями, если они возникают.

Дополнительное задание

Был пройден курс https://learngitbranching.js.org/. Результат выполнения представлен в приложении 1:

- Вместо пункта с SVN было выполнено следующее задание:

  3.1. Опубликовать работу на удаленный репозиторий (github/gitlab), быть готовым продемонстрировать разницу между командам (+ключам): pull, fetch, push. В процессе изучения команд, склонировать работу в новую копию локального репозитория. В дальнейшем (в пункта 3.2-3.x) работать с ней, отдельно должен

быть сохранена директория со всеми ревизиями в рамках первой часть работы для упрощения проверки задания.

3.2. Выполнить временное сохранение незавершённых изменений в рабочем каталоге с последующим восстановлением этих изменений после получения актуального состояния репозитория.

3.3. Выполнить сохранение незавершённых изменений при переключении на другую задачу и последующее применение сохранённых изменений в исходной ветке.

Для 3.2 - 3.3. понадобятся git stash, git switch и их ключи/подкоманды (apply и тд).

Далее потребуется научиться работать с git submodule

3.4. Подключить внешний репозиторий в качестве подмодуля и зафиксировать изменения в основном проекте.

3.5. Обновить подмодуль до новой версии и зафиксировать изменение ссылки на внешний репозиторий.

3.6. Выполнить изменение внутри подмодуля и синхронизировать его с основным проектом.

3.7. Смоделировать работу двух пользователей с подмодулем и выполнить синхронизацию версий.

3.8. Смоделировать конфликт, связанный с различными версиями подмодуля, и выполнить его разрешение.

Действия с submodule комментировать и указывать в отчете по ЛР!

Также для защиты требуется ознакомиться, попробовать в действии и отразить в отчете (на примере любого вашего кода любой лабораторной работы любой дисциплины) следующее:

1. Git hooks; можно добавить hook-и на любую допустимую операцию.
2. git worktree - кратко показать создание второго рабочего дерева;
3. git sparse-checkout - кратко показать частичное извлечение только части проекта.

Список команд для выполнения основного задания:

```
    mkdir src
    cd src
    git init
    git branch -M main
    unzip ../commits/commit0.zip
    git add .
    git commit -m "init commit r0"

    git checkout -b blue1
    unzip -o ../commits/commit1.zip
    git add .
    git  commit  --author  "blue  <blue@example.com>"  -m  "make  branch
blue1 (r1)"

    git checkout -b blue2
    unzip -o ../commits/commit2.zip
    git add .
    git  commit  --author  "blue  <blue@example.com>"  -m  "make  branch
blue2 (r2)"
    unzip -o ../commits/commit3.zip
    git add .
    git commit --author "blue <blue@example.com>" -m "r3"

    git checkout blue1
    unzip -o ../commits/commit4.zip
    git add .
    git commit --author "blue <blue@example.com>" -m "r4"

    git checkout main
    unzip -o ../commits/commit5.zip
    git add .
    git commit -m "r5"
    unzip -o ../commits/commit6.zip
    git add .
    git commit -m "r6"
    unzip -o ../commits/commit7.zip
    git add .
    git commit -m "r7"
    git checkout main
    unzip -o ../commits/commit8.zip
    git add .
    git commit -m "r8"

    git checkout blue1
    unzip -o ../commits/commit9.zip
    git add .
    git commit --author "blue <blue@example.com>" -m "r9"
    unzip -o ../commits/commit10.zip
    git add .
    git commit --author "blue <blue@example.com>" -m "r10"
```

```
    git checkout main
    git merge blue1 --no-commit
    unzip -o ../commits/commit11.zip
    git add .
    git commit -m "Merge branch 'blue1' into 'main' (r11)"

    git checkout blue2
    unzip -o ../commits/commit12.zip
    git add .
    git commit --author "blue <blue@example.com>" -m "r12"

    git checkout main
    git merge blue2 --no-commit
    unzip -o ../commits/commit13.zip
    git add .
    git commit --author "blue <blue@example.com>" -m "Merge branch
'blue2' into 'main' (r13)"
    unzip -o ../commits/commit14.zip
    git add .
    git commit -m "r14"

    git log --graph --oneline
```

Результат выполнения

```
* b5be146 (HEAD -> master) r14
*   25285a5 Merge branch 'blue2' into 'master' (r13)
|\
| * c4abb7b (blue2) r12
| * 9b3a8c3 r3
| * e9e2314 make branch blue2 (r2)
* |   fe395e0 Merge branch 'blue1' into 'master' (r11)
|\ \
| * | 2fdae62 (blue1) r10
| * | 8b07cc8 r9
| * | ec03e9f r4
| |/
| * 223c67d make branch blue1 (r1)
* | 08fa0ac r8
* | 691bf50 r7
* | 474facf r6
* | 793eefb r5
|/
* 2857deb init commit r0
```

Дополнительное задание.

3.1 Опубликовал работу на удаленный репозиторий, предварительно создав репозиторий на github и привязав локальный репозиторий к удалённому.

```
git remote add origin https://github.com/Dkanil/OPI-lab2.git
git branch -M main
git push origin --all
```

Склонировал работу в новую копию локального репозитория.

```
git clone https://github.com/Dkanil/OPI-lab2.git
```

3.2 Временное сохранение незавершённых изменений в рабочем каталоге с последующим восстановлением этих изменений после получения актуального состояния репозитория

Сначала “имитируем” изменения в удалённом репозитории.

```
echo testtesttest > test
git add .
git commit -m "some commit for trying stash"
git push
```

Переходим в другую копию репозитория и там пробуем git stash. (создаем изменения, сохраняем их, качаем удалённый репозиторий, возвращаем изменения с удалением из стэша)

```
echo test > forTask2
git stash -u
git pull
git stash pop
```

3.3 Сохранение незавершённых изменений при переключении на другую задачу и последующее применение сохранённых изменений в исходной ветке

Создаем изменения, сохраняем их, переключаемся на другую ветку и там создаём другие изменения, возвращаемся к прошлой задаче и возвращаем изменения с удалением из стэша)

```
echo test > forTask3
git stash -u
git switch blue1
echo test > anotherTask
git commit -m "some commit in anotherTask"
git switch main
git stash pop
```

3.4. Подключить внешний репозиторий в качестве подмодуля и зафиксировать изменения в основном проекте.

Создаем внешний репозиторий с любым кодом и привязываем его к основному

```
    git init
    git add .
    git commit -m "init commit"
    git remote add origin https://github.com/Dkanil/submodule-OPI-lab2.git
    git branch -M main
    git push --set-upstream origin main
    cd ../src

    git submodule add https://github.com/Dkanil/submodule-OPI-lab2
    git commit -m "add submodule"
    git push
```

3.5. Обновить подмодуль до новой версии и зафиксировать изменение ссылки на внешний репозиторий.

Сымитируем изменения в подмодуле, зальём новую версию в удалённый репозиторий и обновим версию подмодуля в основном репозитории.

```
    cd ../submodule-src
    git add .
    git commit -m "fix: add checking for negative numbers :)"
    git push
    cd ../src

    git submodule update --remote
    git add .
    git commit -m "add changes from submodule"
    git push
```

`git submodule update --remote` выполняет `fetch` и `merge` для субмодуля, “как бы такой аналог `git pull`“

3.6. Выполнить изменение внутри подмодуля и синхронизировать его с основным проектом.

Поменяем код внутри подмодуля, привязанного к основному проекту и запушим эти изменения в эту “стороннюю библиотеку”

```
cd submodule-OPI-lab2
git checkout main
git add .
git commit -m "feat: add input tip"
git push
cd ..

git add .
git commit -m "submodule upd2"
git push
```

3.7. Смоделировать работу двух пользователей с подмодулем и выполнить синхронизацию версий.

Создадим изменения в подмодуле в качестве user2 и запушим их

```
cp -r src src-copy-for-submodule
cd src-copy-for-submodule/submodule-OPI-lab2
echo print\(help\) > help.py
git add .
git commit --author "user2 <no@example.com>" -m "help"
git push

cd ..
git add .
git commit --author "user2 <no@example.com>" -m "help add"
git push
```

Теперь в качестве основого пользователя скачаем эти изменения

```
cd ../src
git pull
```

Здесь если пропишем `git status`, то увидим, что изменения в подмодуле не применились, чтобы это исправить выполним следующую команду:

```
git submodule update --init --recursive
```

(Флаги `--init` и `--recursive` следует писать по советы из документации на случай, если другими пользователями были подключены новые подмодули и есть вложенные подмодули соответственно)

3.8. Смоделировать конфликт, связанный с различными версиями подмодуля, и выполнить его разрешение.

Сымитируем какие-то изменения у одних и тех же файлов от лица разных пользователей в разных копиях подмодуля (как отдельный репозиторий и как подключенный) и попробуем запушить эти изменения.

```
    cd submodule-src
    # создали изменения в help.py и baobab.py
    git add .
    git commit --author "user2 <user2@example.com>" -m "fix help, add
comments in baobab"
    git push


    cd ../src-copy-for-submodule/submodule-OPI-lab2
    # создали изменения в help.py и baobab.py
    git add .
    git commit -m "upd"
    git pull
```

Git здесь начинает ругаться на конфликт версий при слиянии и просит вручную разрешить его.

```
$ git pull
remote: Enumerating objects: 7, done.
remote: Counting objects: 100% (7/7), done.
remote: Compressing objects: 100% (2/2), done.
remote: Total 4 (delta 1), reused 4 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (4/4), 358 bytes | 44.00 KiB/s, done.
From https://github.com/Dkanil/submodule-OPI-lab2
   c0b9890..be8bc14  main       -> origin/main
Auto-merging baobab.py
Auto-merging help.py
CONFLICT (content): Merge conflict in help.py
Automatic merge failed; fix conflicts and then commit the result.
```

Теперь если мы заглянем в файл help.py, то в нём будет две версии кода: наша, и на удаленном репозитории. Сохраняем только одну из них, при том, чтобы остальные файлы оставались рабочими в новой версии, чтобы разрешить конфликт.

```
<<<<<<< HEAD
help = "ne pomogy"
print(help)
=======
print("help")
>>>>>>> be8bc1451b786ea04aee03815d50bcd1c98b7d03
```

```
    git add .
    git commit -m "fix merge conflict"
    git push
```

4. hooks

Создадим в директории `.git/hooks` файл `pre-commit` со следующим содержимым:

```
#!/bin/sh
if git diff --cached | grep '^+' | grep -iq 'fixme'; then
  echo "ERROR: Found 'fixme' in your changes!"
  exit 1
fi
```

Этот “хук” будет выполняться перед коммитом и проверять, есть ли в файлах, которые мы пытаемся закоммитить, пометка `'fixme'`

Теперь добавим в случайный файл комментарий о некорректности кода и попробуем выполнить `git commit`.

```
User@Laptop-DanilKo MINGW64 ~/Desktop/ITMO/semester4/ОПИ/lab2/submodule-src (main)
$ git commit -m "add fixme message"
ERROR: Found 'fixme' in your changes!
```

Наш hook сработал, но если мы всё равно хотим выполнить коммит, игнорируя хуки, то можем добавить флаг `--no-verify`.

5. git worktree

Создадим второе рабочее дерево нашего основного репозитория в состоянии ветки `blue2` и перейдем в него

```
git worktree add ../hotfix-in-blue2 blue2
cd ../hotfix-in-blue2
```

Таким образом мы смогли избежать сохранения в stash большого числа текущих изменений и просто получили два рабочих дерева, в одном из которых состояние репозитория на ветке `main`, а в другой – `blue2`.

6. git sparse-checkout

Попробуем аккуратно “позаимствовать” чужую лабораторную работу 2 по программированию за 1 семестр. Для этого склонируем в новую директорию удалённый репозиторий, с которого хотим украсть только 1 лабу.

(Сначала пытался прописать просто `git clone --no-checkout`, но git качал абсолютно всю историю изменений, которая огромная, тогда добавил `--depth 1`, но это было опять долго. Поэтому добавил флаг `--filter=blob:none`, который не скачивает

содержимое файлов вообще и флаг `--sparse`, который сразу включает режим `sparse-checkout`)

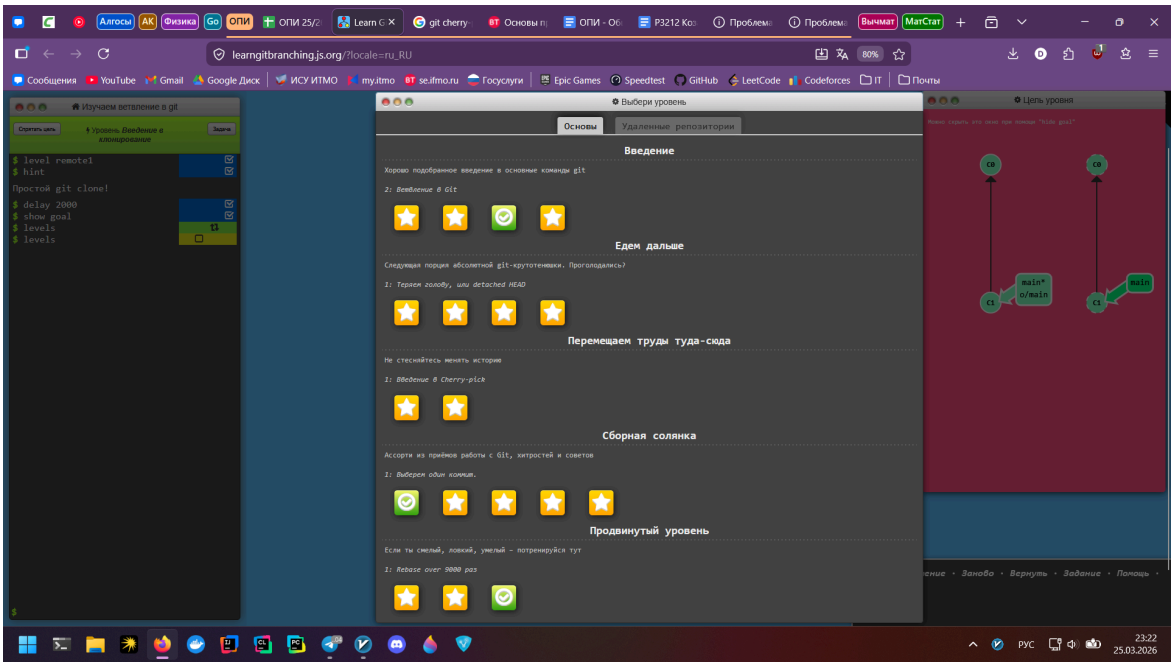
```
    mkdir barsukov
    cd barsukov
    git clone --filter=blob:none --sparse
https://github.com/maxbarsukov/itmo.git
    cd itmo
    git sparse-checkout set "1-2 программирование/лабораторные/lab2"
```

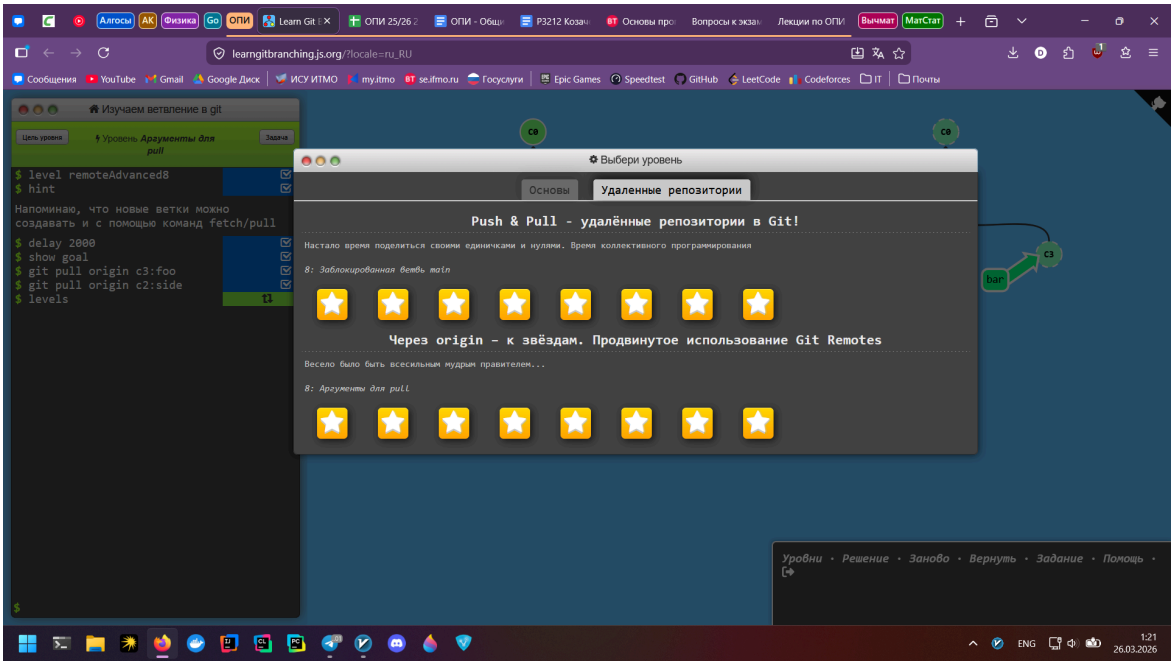
Любуемся “позаимствованной” лабораторной.

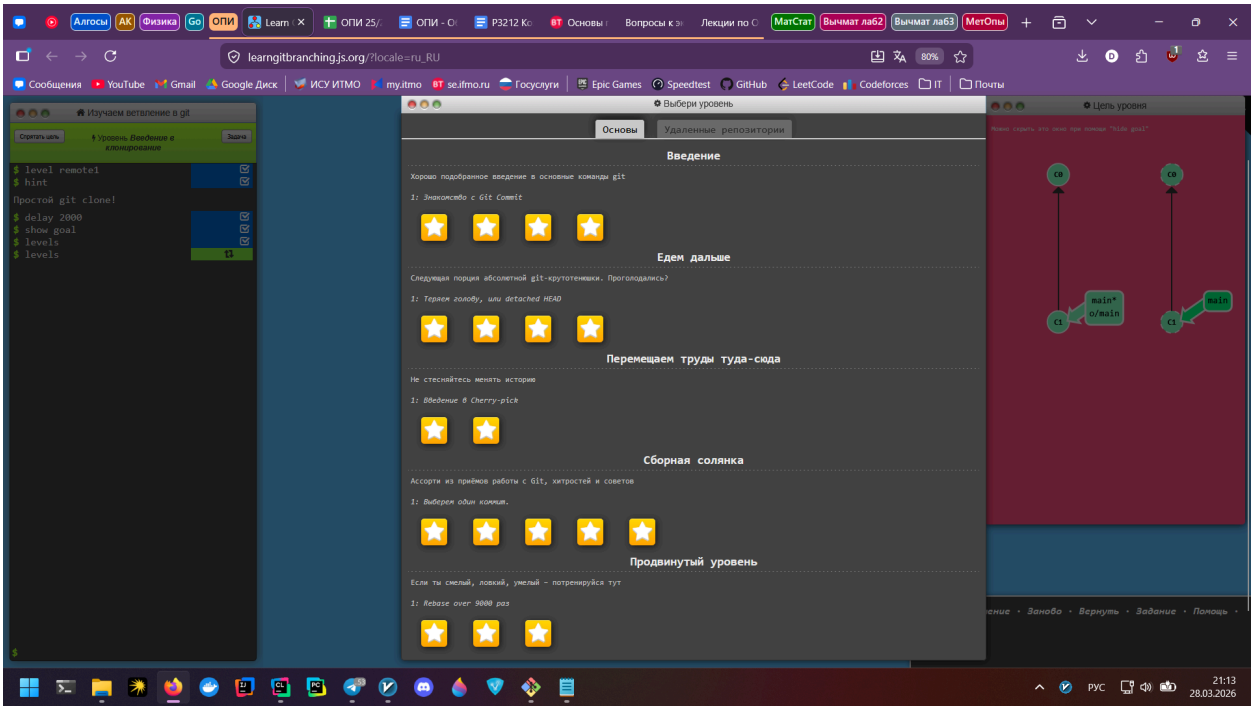
Выводы по работе

В ходе работы были изучены различные команды и их флаги системы контроля версий git, были рассмотрены возможные конфликты слияния и способы их разрешения.

Приложение 1






learngitbranching.js.org/?locale=ru_RU
Изучаем ветвление в git
Выбери уровень
Основы
Удаленные репозитории
Введение
Хорошо подобранное введение в основные команды git
1: Знакомство с Git Commit
Едем дальше
Следующая порция абсолютной git-крутотенюшки. Проголодались?
1: Теряем голову, или detached HEAD
Перемещаем труды туда-сюда
Не стесняйтесь менять историю
1: Введение в Cherry-pick
Сборная солянка
Ассорти из приёмов работы с Git, хитростей и советов
1: Выберем один коммит.
Продвинутый уровень
Если ты смелый, ловкий, умелый – потренируйся тут
1: Rebase over 9000 раз
Цель уровня
$ level remote1
$ hint
Простой git clone!
$ delay 2000
$ show goal
$ levels
$ levels
main*
o/main
main